

Day4PracticeSheet—PandasDataFrames&Filtering

Module: NumPy&Pandas—PandasDataFrames&Filtering

PreparedB Dr.RavneetKaur,Mr.Akarsh

Objective: PractisecreatingDataFrames,loadingCSVs,usinginfo()/describe()),loc,iloc,andbooleanfiltering.

SAMPLEDATASET

Use the Students dataset below (copy into a CSV file named 'students.csv') OR create it directly as a DataFrame using `pd.DataFrame(dict)`.

StudentID	Name	Age	City	Department	Score	Grade	Passed
S001	ArjunMehta	21	Delhi	CSE	78	B	TRUE
S002	DivyaSharma	22	Mumbai	ECE	91	A	TRUE
S003	KaranPatel	20	Pune	CSE	55	C	TRUE
S004	MeeraNair	23	Chennai	MECH	42	D	TRUE
S005	SahilGupta	21	Delhi	ECE	88	A	TRUE
S006	AnjaliSingh	22	Jaipur	CSE	35	F	####
S007	RohanVerma	20	Mumbai	MECH	67	B	TRUE
S008	PoojaYadav	23	Chennai	CSE	92	A	TRUE
S009	VivekKumar	21	Pune	ECE	71	B	TRUE
S010	NishaAgarwal	22	Delhi	MECH	58	C	TRUE

PRACTICEQUESTIONS

Question	TaskDescription	YourAnswer/Code
----------	-----------------	-----------------

Q1[Load&Inspect]

Create the DataFrame from the table above. Display: shape, columns, dtypes, head(5), and tail(3).

```
import pandas as pd
import numpy as np

data = {
    "StudentID": ["S0001", "S0002", "S0003", "S0004", "S0005", "S0006", "S0007", "S0008", "S0009", "S0010"],
    "Name": ["Arjun Mehta", "Divya Sharma", "Karan Patel", "Meera Nair", "Sahil Gupta",
            "Anjali Singh", "Rohan Verma", "Pooja Yadav", "Vivek Kumar", "Nisha Agarwal"],
    "Age": [21, 22, 20, 23, 21, 22, 20, 23, 21, 22],
    "City": ["Delhi", "Mumbai", "Pune", "Chennai", "Delhi", "Jaipur", "Mumbai", "Chennai", "Pune", "Delhi"],
    "Department": ["CSE", "ECE", "CSE", "MECH", "ECE", "CSE", "MECH", "CSE", "ECE", "MECH"],
    "Score": [78, 81, 85, 82, 88, 85, 87, 92, 71, 88],
    "Grade": ["B", "A", "C", "D", "A", "B", "B", "A", "B", "C"],
    "Passed": [True, True, True, True, True, False, True, True, True, True]
}

df = pd.DataFrame(data)

print("Shape:", df.shape)
print("Columns:")
print(df.columns)

print("\nData Types:")
print(df.dtypes)
```

```
Shape: (10, 8)
--
Columns:
Index: ['StudentID', 'Name', 'Age', 'City', 'Department', 'Score', 'Grade',
       'Passed'],
      dtype: object

Data Types:
StudentID    object
Name         object
Age          int64
City         object
Department   object
Score        int64
Grade        object
Passed       bool
dtypes: object

First 5 Rows:
StudentID  Name  Age  City  Department  Score  Grade  Passed
0  S0001  Arjun Mehta  21  Delhi  CSE  78  B  True
1  S0002  Divya Sharma  22  Mumbai  ECE  81  A  True
2  S0003  Karan Patel  20  Pune  CSE  85  C  True
3  S0004  Meera Nair  23  Chennai  MECH  82  D  True
4  S0005  Sahil Gupta  21  Delhi  ECE  88  A  True

Last 3 Rows:
StudentID  Name  Age  City  Department  Score  Grade  Passed
7  S0008  Pooja Yadav  23  Chennai  CSE  82  A  True
8  S0009  Vivek Kumar  21  Pune  ECE  71  B  True
9  S0010  Nisha Agarwal  22  Delhi  MECH  88  C  True
```

Q2
[describe/info]

Run df.info() and df.describe(). From describe(): what is the mean Score? What is the standard deviation?

```
df.info()

print("\n")

print(df.describe())

print("\nMean Score =", df["Score"].mean())

print("Standard Deviation =", df["Score"].std())

--
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 8 columns):
 #   Column          Non-Null Count  Dtype
---  -
 0   StudentID      10 non-null    object
 1   Name           10 non-null    object
 2   Age            10 non-null    int64
 3   City           10 non-null    object
 4   Department     10 non-null    object
 5   Score          10 non-null    int64
 6   Grade          10 non-null    object
 7   Passed         10 non-null    bool
dtypes: bool(1), int64(2), object(5)
memory usage: 702.0+ bytes

      Age      Score
count  10.000000  10.000000
mean     21.500000  67.700000
std       1.080123  20.133168
min      20.000000  35.000000
25%      21.000000  55.750000
50%      21.500000  69.000000
75%      22.000000  85.500000
max      23.000000  92.000000
```

Q3 [Column Access]	Access the 'Name' column using df['Name']. Then access 'Score' and 'Grade' columns together using double brackets.	<pre>print(df["Name"]) print("\n") print(df[["Score", "Grade"]])</pre> <pre>0 Arjun Mehta 1 Divya Sharma 2 Karan Patel 3 Meera Nair 4 Sahil Gupta 5 Anjali Singh 6 Rohan Verma 7 Pooja Yadav 8 Vivek Kumar 9 Nisha Agarwal Name: Name, dtype: object</pre> <pre> Score Grade 0 78 B 1 91 A 2 55 C 3 42 D 4 88 A 5 35 F 6 67 B 7 92 A 8 71 B 9 58 C</pre>
Q4[loc]	Use df.loc to select: (a) row at index 0, (b) rows 2 to 5, (c) rows 0–4 but only columns Name, Score, Grade.	<pre># (a) print(df.loc[0]) # (b) print(df.loc[2:5]) # (c) print(df.loc[0:4, ["Name", "Score", "Grade"]])</pre> <pre>StudentID S001 Name Arjun Mehta Age 21 City Delhi Department CSE Score 78 Grade B Passed True Name: 0, dtype: object</pre> <pre>StudentID Name Age City Department Score Grade Passed 2 S003 Karan Patel 20 Pune CSE 55 C True 3 S004 Meera Nair 23 Chennai MECH 42 D True 4 S005 Sahil Gupta 21 Delhi CSE 88 A True 5 S006 Anjali Singh 22 Jaipur CSE 35 F False</pre> <pre> Name Score Grade 0 Arjun Mehta 78 B 1 Divya Sharma 91 A 2 Karan Patel 55 C 3 Meera Nair 42 D 4 Sahil Gupta 88 A</pre>
Q5[iloc]	Use df.iloc to select: (a) first 3 rows, (b) last 2 rows, (c) rows at positions 1, 3, 5 and columns at positions 0, 2, 4.	<pre># (a) print(df.iloc[:3]) # (b) print(df.iloc[-2:]) # (c) print(df.iloc[[1,3,5],[0,2,4]])</pre> <pre>... StudentID Name Age City Department Score Grade Passed 0 S001 Arjun Mehta 21 Delhi CSE 78 B True 1 S002 Divya Sharma 22 Mumbai ECE 91 A True 2 S003 Karan Patel 20 Pune CSE 55 C True</pre> <pre>StudentID Name Age City Department Score Grade Passed 8 S009 Vivek Kumar 21 Pune ECE 71 B True 9 S010 Nisha Agarwal 22 Delhi MECH 58 C True</pre> <pre>StudentID Age Department 1 S002 22 ECE 3 S004 23 MECH 5 S006 22 CSE</pre>
Q6 [Boolean Filter]	Filter students where Score > 80. How many students scored above 80?	<pre>result = df[df["Score"]>80] print(result) print(f"\nStudents Scored Above 80 = ", len(result))</pre> <pre>StudentID Name Age City Department Score Grade Passed 1 S002 Divya Sharma 22 Mumbai ECE 91 A True 4 S005 Sahil Gupta 21 Delhi ECE 88 A True 7 S008 Pooja Yadav 23 Chennai CSE 92 A True</pre> Students Scored Above 80 = 3

Q7 [BooleanFilter]	Filter students from Delhi who are in CSE department. ShowName, Score, and Grade only.	<pre> result = df[(df["City"]=="Delhi") & (df["Department"]=="CSE")] print(result[["Name", "Score", "Grade"]]) </pre> <pre> ... Name Score Grade 0 Arjun Mehta 78 B </pre>
Q8 [BooleanFilter]	Find all students who Failed (Passed == False). What is their averageScore?	<pre> failed = df[df["Passed"]==False] print(failed) print("\naverage Score = ", failed["Score"].mean()) </pre> <pre> ... StudentID Name Age City Department Score Grade Passed 5 5006 Anjali Singh 22 Jaipur CSE 35 F False Average Score = 35.0 </pre>
Q9[StringFilter]	Filter students whose Name contains the string 'a' (case-insensitive) using .str.contains().	<pre> result = df[df["Name"].str.contains("a",case=False)] print(result) </pre> <pre> ... StudentID Name Age City Department Score Grade Passed 0 5001 Arjun Mehta 21 Delhi CSE 78 B True 1 5002 Divya Sharma 22 Mumbai ECE 91 A True 2 5003 Karan Patel 20 Pune CSE 55 C True 3 5004 Meera Nair 23 Chennai MECH 42 D True 4 5005 Sahil Gupta 21 Delhi ECE 88 A True 5 5006 Anjali Singh 22 Jaipur CSE 35 F False 6 5007 Rohan Verma 20 Mumbai MECH 67 B True 7 5008 Pooja Yadav 23 Chennai CSE 92 A True 8 5009 Vivek Kumar 21 Pune ECE 71 B True 9 5010 Nisha Agarwal 22 Delhi MECH 58 C True </pre>
Q10[Add Column]	Add a new column 'Score_Category': if Score >= 80 → 'Distinction', Score >= 60 → 'Merit', else 'Pass/Fail'. Use np.where() or nested condition.	<pre> df["Score_Category"] = np.where(df["Score"]>=80, "Distinction", np.where(df["Score"]>=60, "Merit", "Pass/Fail")) print(df) </pre> <pre> ... StudentID Name Age City Department Score Grade Passed \ 0 5001 Arjun Mehta 21 Delhi CSE 78 B True 1 5002 Divya Sharma 22 Mumbai ECE 91 A True 2 5003 Karan Patel 20 Pune CSE 55 C True 3 5004 Meera Nair 23 Chennai MECH 42 D True 4 5005 Sahil Gupta 21 Delhi ECE 88 A True 5 5006 Anjali Singh 22 Jaipur CSE 35 F False 6 5007 Rohan Verma 20 Mumbai MECH 67 B True 7 5008 Pooja Yadav 23 Chennai CSE 92 A True 8 5009 Vivek Kumar 21 Pune ECE 71 B True 9 5010 Nisha Agarwal 22 Delhi MECH 58 C True Score_Category 0 Distinction 1 Distinction 2 Pass/Fail 3 Pass/Fail 4 Distinction 5 Pass/Fail 6 Distinction 7 Distinction 8 Merit 9 Pass/Fail </pre>
BONUS	Sort the DataFrame by Score in descending order. Reset the index. Show the top 5 students.	<pre> sorted_df = df.sort_values(by="Score", ascending=False).reset_index(drop=True) print(sorted_df.head()) </pre> <pre> ... StudentID Name Age City Department Score Grade Passed \ 0 5008 Pooja Yadav 23 Chennai CSE 92 A True 1 5002 Divya Sharma 22 Mumbai ECE 91 A True 2 5005 Sahil Gupta 21 Delhi ECE 88 A True 3 5001 Arjun Mehta 21 Delhi CSE 78 B True 4 5009 Vivek Kumar 21 Pune ECE 71 B True Score_Category 0 Distinction 1 Distinction 2 Distinction 3 Merit 4 Merit </pre>